

Autonomous Navigation and Parking of a Robotic Car

AAP-RAS Project Report

Group 2

Sumit Patidar, Alexandros Nikolaou, Sharang Kaul, Janis Freund, Jhorman Perez, Stoyan Karastoyanov

Abstract—This report presents a few methodologies and observations from a robotic car with autonomous features in the MORSE simulation environment using ROS. It presents the framework and system architecture used to provide autonomous driving capabilities to the car in a simulation environment. Challenges of perception, prediction, and autonomous driving behavior are handled in this project. The main objective is to drive the robotic car autonomously and park in the identified parking spot in the simulation environment while following the constraints. The valuable part of research lies in detecting static, dynamic objects and road lanes and creating a dynamic map based on projections for motion planning. The research work also includes implementing an online prediction module using RNNs, tuning a Time Elastic Band (TEB) local planner, and a finite state machine-based decision-maker to park the car autonomously. The report details the system integration and measurements from the experiments and analyzes the results in the form of an evaluation matrix. Further, analysis and discussion are performed for each sub-module. These results are initial observations, and further improvements are possible for a robust implementation of autonomous driving and parking application.

Index Terms—Object Detection (YOLOv4), Lane Detection and Segmentation (LaneNet), Prediction (RNNs), Motion Planning (TEB), Decision Making (FSMs)

I. INTRODUCTION

New technologies in communication and robotics have had a substantial influence on our daily lifestyle. These technologies have given rise to the prospect of autonomous robots being used today in private households (vacuum cleaners, lawnmowers), for military reconnaissance in rugged terrain, in driverless cars and drones. In this era, driven by technological and socioeconomic factors, transportation has evolved to a point where a further transition to driverless vehicles seems inevitable. Autonomous vehicles seemed to be more reliable, convenient and intended to eliminate the stress and fatigue of drivers [1]. Moreover, if these vehicles can act as agents and collaborate, they can plan, drive and explore more efficiently. Problems such as the complexity of perception, effective

control without endangering the environment, a lack of learning aptitude and adaptability continue to confront researchers with daunting future challenges. Thus, an understanding of autonomous systems remains essential core of the trending research.

There are plenty of challenges to autonomous driving, but in this report, we focus on three main challenges: perception, prediction, and advanced autonomous driving behavior. To that end, a small-scale car, a track with lanes, static and dynamic objects, and a set of traffic signs are provided in a Morse simulation environment. At the end of the project, the goal is for this car is to traverse the track with lanes, avoid the static and moving obstacles, properly follow the traffic signs, and park into a designated spot, all of that - autonomously.

The research in the domain of autonomous driving is evolving at such a tremendous pace, resulting in various approaches to tackle the challenges. The SLAM domain has made quick progress from filter-based to optimization-based techniques. The perception is heavily dominated by deep convolutional networks (DCNNs) and modern computer vision techniques. Even to predict the moving objects, recurrent neural network (RNN) models form state-of-the-art techniques. Reinforcement learning is widely used for complex maneuvering, model predictive control (MPC) for planning and control, and behavior trees for decision making.

For this project, we borrowed a few state-of-the-art approaches for perception and prediction tasks, i.e. using DNNs and post-processing using computer vision techniques. We ended up using RNNs for prediction, Time Elastic Band (TEB) planner for local motion planning, and finite-state-machines for the decision-making unit. We used specific useful quantitative metrics to demonstrate the performance of chosen approaches and compare them against other approaches. And there are also qualitative results in the form of images, graphs, and plots resulting from the simulation environment.

We discuss the above approaches and related work for SLAM, perception, prediction, motion planning, and decision making in more detail in the following section.

II. RELATED WORK

Perception, prediction and advanced driving capabilities are the critical challenges for our project module towards autonomous navigation.

SLAM: Estimation can be separated into two categories: Filter-based and Optimization-based SLAM [2]. EKF SLAM and Particle Filter SLAM are the two main methods used in the Filter-based category. Whereas Graph-based SLAM is leading the way in the Optimization-based SLAM. More recent developments use vision-based SLAM approaches with ORBSLAM2 [3] and GCNv2 [4] being the most efficient in the field. For our simulation scenario, hector_slam performs well in all use case scenarios. hector_mapping is a SLAM approach that can be used with or without odometry and on platforms that exhibit roll/pitch motion.

Perception: For the perception module, the state-of-the-art methods all rely on Deep Convolutional Neural Networks (DCNNs). There currently exists a clear distinction in them: Single- and two-stage detection frameworks. The former use a single network to produce detected objects locations and class predictions simultaneously, while the latter use region proposal networks to generate at the first stage and then categorize regions of interest at the second stage. For our simulation, we decided to test the single-stage detection algorithms *You Only Look Once (YOLO)* [5] and *Single Shot Detector (SSD)* [6] since they operate a lot faster in real-time, to achieve competitive results. For the specific needs of our simulation, we pursued and implemented a single-stage detection framework (YOLOv4) considering both the computational cost and the accuracy required [7].

Traditional lane detection methods follow a feature-based approach where low-level edge features are used to identify lane lines. Model-based approaches instead rely on "common rules" for lane configurations such as straight lines remaining in parallel and parallel lines meet in a vanishing point. One important method to filter straight lines from a noisy image is *Linear Hough Transform* (e.g. in [8]). However, these methods cannot distinguish between lane lines and other lines in the image. Data-driven approaches based on deep neural networks can handle these situations better. They learn patterns from images with labeled ground truth features called supervised classification. These approaches require a lot of labeled data that can be used for training a model. Because of this demand there exist several annotated datasets such as *CULane* [9], *TuSimple* [10], *CalTech* [11], and *BDD* [12]. A promising approach based on deep neural networks is called *VPGNet* [13]. It combines the advantages of deep learning with prior knowledge about the geometry of the road. Another approach is the *LaneNet* model [14]. It formulates lane detection as an instance segmentation problem where each lane corresponds to a different instance of the lane class.

Prediction: A self-driving car needs to project the future behaviour of obstacles around to react appropriately to avoid

accidents by providing additional knowledge for the motion planner. Several methods are intending to tackle prediction, such as [15], where the technique followed with recurrent network models allows to have a multi-agent behaviour prediction model, accounting dynamic objects and relying extensively on heterogeneous datasets, resulting in a generally-applicable approach for real-world scenarios forecasting. Lyft develops another example in [16], also supported by vast datasets of multiple types from LiDAR, camera, and radar sensors to enable motion prediction. Recurrent Neural Network (RNN) models are used again in [17] for real-time dynamic obstacle motion prediction, given the sequential nature of the input we can have from the objects' state in time. The model is based on recent observations of the obstacles for training and prediction in future time steps, and it is suitable for non-linear motion. This last one is the approach that seems more suitable for this project, appearing to be the lightest and most efficient solution given our specific environment.

Motion Planning: The objective of the local planner is to execute a global plan without fail. In other words, in order to complete its path, the robotic car must find trajectories to avoid obstacles and satisfy optimization criteria in the configuration space (C-space), given a starting and goal point. A detailed local planning review is presented in [18] where the taxonomy of motion planning was divided into four groups; graph-based planners, sampling-based planners, interpolating curve planners and numerical optimization approaches. After a summary of these conventional planners, the emerging deep learning-based planners are the latest state of the art. Nevertheless, for the MORSE environment, the *teb_local_planner* implementing a plugin to the *base_local_planner* of the 2D navigation stack is computationally inexpensive and performs well.

Decision Making: The handling of decision making for intelligent systems is achieved through the Finite State Machines (FSM) planner [19]. However, FSMs are prone to modularity, leading to introducing the behavior trees technique as an alternative. Advanced state machines like Behavior trees (BTs) [20] increases modularity, simplifying the way human and machine design. As the number of scenarios and behaviours increases, the finite state machine that is needed becomes significantly more complex, with many more states and conditions for transition. Still, it is an excellent starting point for systems with restricted operational design domains, permitting a manageable number of states.

III. SYSTEM COMPONENTS

A. System Requirements

The requirements of our system are as follows:

- **Perception:** The requirements from the perception module are to recognize the traffic signs (turn right/left, go straight, construction area, slipperiness, stop, parking, parking slot), objects (static and dynamic), and road lanes in the environment.

- **Prediction:** The car shall predict the future behavior of traffic entities in the environment i.e. predict the trajectories of moving entities.
- **Dynamic mapping:** The perceived objects in the environment such as the road lanes shall be projected on a 2d-costmap. Also, the static objects shall be visualized on the dynamic map in the form of markers. Finally, the predicted trajectories of the moving obstacles through the prediction module shall be projected on the dynamic costmap.
- **Parking:** The car shall be able to find an available spot to park autonomously or provided the parking station location.
- **Decision making:** The car should be able to make decisions based on its status and change its states accordingly. For example: if the battery is running low, the car should go to the charging station, recharge, and continue its previous goal.

B. System Architecture

The system architecture for our project is shown in figure 1. The main components we have worked on includes behaviour_planner, perception, prediction, costmap_2d, and teb_local_planner. The system starts with a goal point (pose) as an input or parking as a goal. The behavioral planner is responsible for executing the goal taking into account the status of the car. If the car is running low on battery, then the planner prioritizes the recharging of the car by setting the goal point to the nearest charging station. The goal points from the behavior planner are passed to the global planner who is responsible for generating the global plan based on the generated dynamic 2D layered costmap. The global and local costmaps are generated by projecting the perceived environment (static and moving objects, road-lane, etc) on an occupancy grid.

The perception module has three sub-modules to perceive the environment. The object detection sub-module is responsible for detecting objects in the morse environment such as traffic signs, objects, etc whereas semantic mapping is used for visualizing the detected objects on the map in the form of rviz markers. The lane detection package is doing the task of segmenting the road lanes and projecting them into a lane costmap. The generated global plan is used by teb_local_planner to generate an optimal local path based on the local costmap. The local planner tries to follow the local path and generates linear and angular velocities commands to drive the car.

The other modules form the core components of the project and were provided as part of the starter code. The covariance_setter module fixes the missing covariance values in the odometry sensor readings, and the ekf_se estimates the robot position. The hector mapping takes in 2D lidar readings and updates the map. The rf2o_laser_odometry takes in laser scan and converts it into odometry readings to be later used in

sensor fusion. Finally, the morse module is responsible for launching the simulation environment, publishing information about the environment and the robotic car, and subscribing to cmd_vel topic to drive the car.

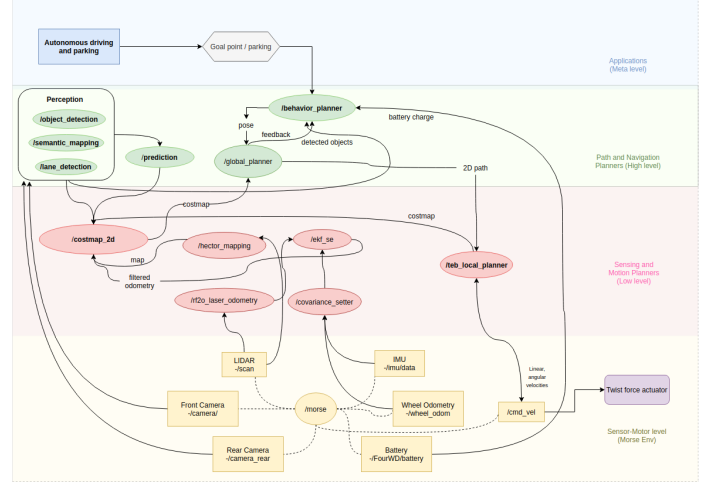


Fig. 1: System architecture

IV. METHODOLOGY

A. Localization and Mapping

Simultaneous Localization and Mapping is a concept that allows a mobile robot to use sensor data to generate a map of an unknown environment while driving, and simultaneously localize its position with-in the map [21]. Thus, localization of the vehicle and map generation are the two primary purposes of SLAM.; all to be executed while driving. No prior knowledge is needed about the map nor about the environment when the mobile robot is navigating. Hence, it is to be considered one of the most important and complex topics to study when developing autonomous robots [22].

Robotic cars have to be considered operating in unknown environments to be more robust against frequent changes. It means the SLAM problem has to be solved to generate sufficiently accurate metric maps useful for navigation of a robot system. For this task, we have been provided with *hector_slam*, consisting of *hector_mapping*, *hector_map_server*, *hector_geotiff* and *hector_trajectory_server* modules. As odometry is notoriously unreliable in real-world scenarios, the system is designed to require fast LiDAR data scan-matching at full LiDAR update rate instead of relying on odometry data from wheel encoders and IMU. Combined with an attitude estimation system and an optional pitch/roll unit to stabilize the laser scanner, the system can provide environment maps even if the ground is non-flat as encountered in the MORSE simulation arena. A comprehensive discussion of hector slam is available in [23].

The estimation of the full six degrees of freedom (robot pose and twist) is realized in the hector pose estimation

node that implements an Extended Kalman Filter (EKF) and fuses measurements from an inertial measurement unit (IMU), wheel encoders, and the 2D pose error from the laser scan matcher. The filter is based on a generic motion model for ground vehicles and is primarily driven by the IMU and wheel odometry. The ‘robot_localization’ package contains an EKF node which is used in this project for all EKF applications by configuring a .yaml text file. Necessary configurations for this file include *sensor types*, *sensor message formats*, *covariance values*, *etc* and more advanced settings, including how differentiation and integration of various sensor readings should occur.

B. Perception

1) **Object Detection:** Considering the specific scenario in the project, we decided to go forth with Single-Stage Detection Frameworks as opposed to Region Proposal Networks ones. The main reason for this is that the Single-Stage Frameworks, by producing object locations and class predictions simultaneously, operate a lot faster in real-time.

The dataset that was used for training and testing was borrowed from the established project’s labelled samples and some small modifications were made to ensure compatibility with our models. The associated classes of the objects are left, right, curve right, construction, slippery, stop, parking, parking spot, human and unknown. The dataset contains 520 images that were labelled using the labelme annotation tool [24]. We also removed all the *lane* labels since these were taken care by the lanes detection module. An in-fly data augmentation procedure was also carried out in order to increase the data diversity without having to collect new samples. The further perception research was divided into two algorithms, namely SSD - Single Shot Detector and YOLO - You Only Look Once. The goal was to use whichever achieved better results and turned out to be more easily integrable into the entire system.

Single Shot Detector (SSD)

The model was taken from TensorFlow’s Model Detection Zoo [25] and uses mobilenet as an underlying architecture. The specific architecture choice was made on the basis of balancing inference speed and precision values as given by TensorFlow. The model was then trained through utilizing the TensorFlow Object Detection API [26]. The metrics for the evaluation of the model were Mean Average Precision at 50% Intersection Over Union threshold and Mean Average Recall for up to 10 detections per image. Results can be seen figures 2 and 3. Further research into SSD was dropped due to better performance of the YOLO system and project time constraints.

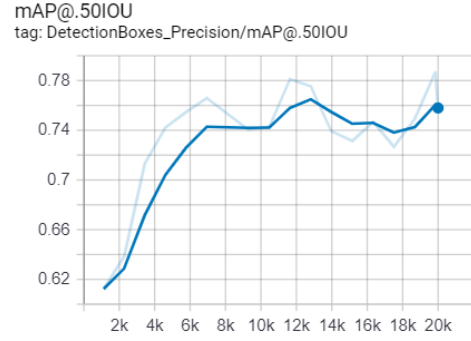


Fig. 2: SSD-Precision

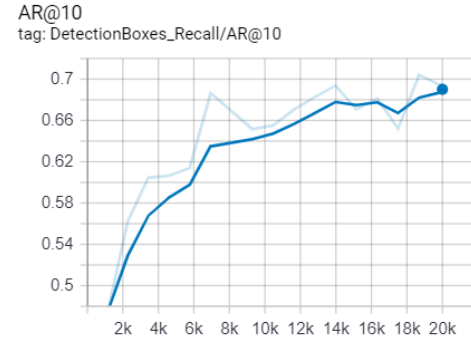


Fig. 3: SSD-Recall

You Only Look Once (YOLOv4)

YOLO [5] object detection method is based on the darknet architecture of CNN. Its characteristics that make it preferable over other methods for our simulation are that it is remarkably fast for real-time processing, the predictions are made from one single network and it better generalizes images than other methods.

For the YOLO object detection module we were based on the *darknet_ros* package [27]. *Darknet_ros* integrates the full YOLOv3. However, we chose to train our simulation environment with YOLOv4 [28], since it is more flexible and parameterized that proved to be valuable for our integration. In particular, we trained the YOLOv4 Tiny model since the simulation environment of our autonomous vehicle is not very demanding, in order to decrease the latency and the computation cost at the highest extent. YOLOv4 is also optimal for real-time object detection tasks because the network lies on the Pareto optimal curve of the AP(accuracy) / FPS(speed) chart.

YOLOv4 model consists of 13 convolutional, 6 max-pool, 2 route, 1 upsample and 2 yolo layers. The model’s characteristics on which it was trained on, as well as the YOLOv4 pre-trained initial weights are shown on the table I.

Test Cases	Distance (m)
Network Size	416 x 416
Learning rate	0.001
Batch	64
Subdivisions	16
Max batches	22.000
Steps	17.600, 19.800
Activation function	Leaky
CUDA/CuDNN	yes
OpenCV	yes
Classes	10
Initial weights	YOLOv4 tiny pre-trained weights [29]

TABLE I: YOLOv4 tiny model characteristics



Fig. 4: YOLOv4 Bounding Boxes

The *darknet_ros* ROS package uses the trained YOLOv4 Tiny model to generate bounding box messages that are published for each detected object, along with their corresponding label and accuracy. An example of the generated bounding boxes to detected objects is shown in figure 4.

2) Lane Detection and Segmentation:

LaneNet - Real Time Lane Detection Networks

The lane detection is based on a CNN called *LaneNet* proposed in [14]. It consists of two branches. The first one is a lane segmentation branch that assigns pixels from a camera image to the two classes lane and background. The second one splits up different lane lines into different instances. We decided to use the *LaneNet* implementation from [30]. The already existing dataset of labeled images was used in a modified form to train the model. First, the polygons labeling the lane lines were replaced because they were inaccurate further away from the robot. Thus, the resulting lanes in the model without the adjustment were thicker than in the original image. One example for the relabeled images is shown in figure 5.

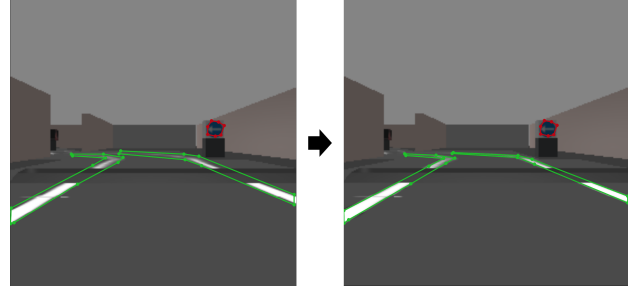


Fig. 5: Relabeling of lane lines

Also, the model sometimes recognized speed bumps as lanes. To solve this problem, we included more labeled speed bump images in the dataset. The relabeling was done using the program *labelme* [24]. Since the chosen *LaneNet* implementation requires the data to be in a certain format, we converted the polygon lane labeling into a binary pixel-wise segmentation with lanes corresponding to the value 255 and background corresponding to 0. Likewise, an instance segmentation image is required as an input that uses different pixel values for different lanes and the value 0 for background. After these adjustments we ended up with 405 images. For the training we used a training-validation ratio of 90:10 with randomly selected images.

We could not achieve good instance segmentation results and therefore we rely on the binary segmentation only for our implementation. To make the trained model available in our ROS environment we integrated the ROS node from [31] which is based on the *LaneNet* implementation from [30] that we used to train the model.

To be able to use the segmentation result for navigating the robot, the detected pixels corresponding to lanes have to be projected into a map. Therefore, we transformed the segmentation image into bird's-eye view by calculating the 3D position of each pixel while setting $z = 0$. The information of both the depth camera and the intrinsic parameters of the main camera is used in order to compute the projection. In the process we also have to take care of correcting the current roll and pitch of the robot. For a segmentation image with the dimensions $v \times u$, equation 1 is implemented for correcting the roll and equation 2 is used to correct the pitch of the robot and therefore of the camera.

$$\begin{aligned} u &= \lfloor \cos(\text{roll}) * u - \sin(\text{roll}) * v \rfloor \\ v &= \lfloor \sin(\text{roll}) * u - \cos(\text{roll}) * v \rfloor \end{aligned} \quad (1)$$

$$v = \lfloor \cos(\text{pitch}) * v \rfloor \quad (2)$$

roll and *pitch* are the corresponding values in radians that we can calculate from the SLAM result of the robot's orientation that is given in quaternions. The actual projection of the points into 3D space is done by using equation 3.

$$\begin{aligned} x &= (u - c_x) * \frac{\text{depth}_{v,u}}{f_x} \\ y &= \text{depth}_{v,u} \end{aligned} \quad (3)$$

c_x is the principal position and f_x the focal length of the camera along the x-axis. $depth_{v,u}$ is the depth information from the depth camera for the corresponding pixel. An example for the translation is shown in figure 6. The left image shows a sample segmentation. After transforming each white lane point to its 3D representation, we get the 2D point cloud as shown in the right image.

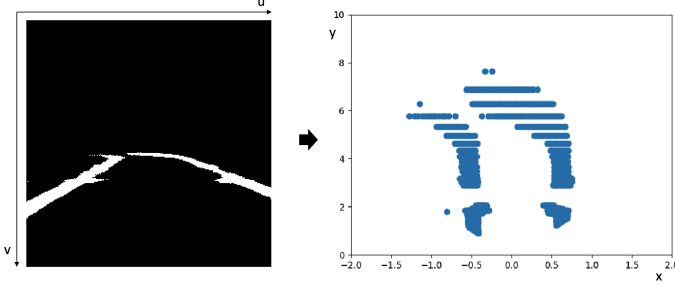


Fig. 6: Transformation of segmentation result to bird's-eye view

After the transformation, the next step is to project the points into a global map. Hence, we need to take the position and yaw of the robot into account. The global costmap consist of a predefined number of discrete cells. To find out to which cell index a certain point (x, y) is projected we are first calculating the base frame angle θ (equation 4) of the point and the distance d from the camera (equation 5).

$$\theta = \text{normalize}(\text{yaw} + \arctan2(y, x) - \arctan2(1, 0)) \quad (4)$$

$$d = \sqrt{x^2 + y^2} \quad (5)$$

The `normalize` function outputs a normalized angle with a value between $-\pi$ and π . Similarly to *roll* and *pitch*, *yaw* is calculated from the SLAM result. The final index is calculated as shown in equation 6.

$$\begin{aligned} idx_x &= \frac{|origin_x - pose_x - d * \cos \theta|}{res} \\ idx_y &= \frac{|origin_y - pose_y - d * \sin \theta|}{res} \\ idx &= [idx_x] + [idx_y] * width \end{aligned} \quad (6)$$

Here, *origin* describes the map origin, *res* the map resolution, and *width* the map width. *pose* is the current pose of the robot.

In the costmap, an obstacle can be simulated by setting the value of a certain cell to 100. Simply setting the value of a cell to 100 after a lane point was mapped to that cell did not produce good results because of disturbances. So we introduced a *hitmap* and a *reference_map*. Both variables have the same number of entries as the global map. The *hitmap* stores the number of times that a lane point was mapped to the corresponding index. For each image received by the *LaneNet* node there is a loop through all pixels, projecting each of the pixels to 3D points and then to a map. The *reference_map* stores the number of times these points have been mapped to the corresponding index. We only set the actual map index

to be occupied if corresponding *hitmap* entry is greater than a threshold and the ratio of the corresponding *hitmap* entry divided by the *reference_map* entry is greater than a threshold ratio. If the sum of the value of the *hitmap* and the value of the *reference_map* is greater than a buffer size, both values get divided by two. Hence, it should not happen that a possible wrong map estimation cannot be overwritten.

In the created map there are gaps in some lane lines, e.g. when there is a speed bump. To tackle this problem we implemented curve fitting on a local version of the costmap. First, all lane lines building connected components and the corresponding points were determined. For each of the connected components we fit a second degree polynomial through all the points. Then, we can calculate both end points of each function. At these points the slope of the function can be computed using the derivative. Since the curves may lie in different coordinate frames, slopes are not necessarily comparable. Therefore we convert slopes into angles with the same reference. Finally, we try to connect all end points to another end point with a straight line if two conditions hold:

- The angles are similar but in the opposite direction
- The Euclidean distance is smaller than a threshold

All found lines are added as lane obstacles to the global costmap. The complete curve fitting pipeline is presented in figure 7.

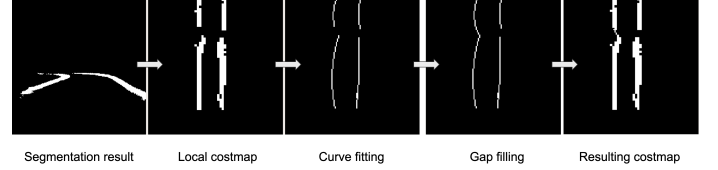


Fig. 7: Curve fitting pipeline

3) 3D Objects Semantic Mapping: Detected objects have to be converted from the 2D camera image into a real-time 3D object representation in a dynamic map in order to facilitate planning and prediction. The main challenge for that is pixel-by-pixel data association with the dynamic map and occlusions. The steps we followed towards our goal were:

- a) Publish depth camera's point cloud information.
- b) Establish a transform listener communication with the YOLO's bounding box messages.
- c) Cluster detected associated points per bounding box as one object using *kdtree*.
- d) Visualize the objects in the dynamic map using markers, either as cubes or cylinders depending on its dimension.
- e) Track and merge identical objects if they refer to the same detected id.
- f) Keep tracking and visualize the static objects in the map and feed the dynamic objects to the prediction module.

For publishing the depth camera's point cloud we used the *freenect* package [32], while for the whole 3D object semantic

package we were based on the [33]. The point cloud were fused with the information we got from the bounding boxes of the *darknet_ros* package, to be translated into a 3D voxel grid in the global map, by using kdTree clustering. To solve the data association problem we made use of a simple tracking approach. First, we only projected 3D objects in the dynamic map if they were below a distance threshold of 5m. Then by comparing the intersection over union (IoU) of the absolute location of the detected objects that belonged to the same class but were generated in different timestamps, we merged them into one particular object, gave them an identity and tracked their average point, if they were projected in a range of identity threshold. *Identity threshold* refers to the difference of the location of two projected objects. If the IoU, translated here as relative distance, was above the *identity threshold* then we considered these objects as one. This approach however lacked in the cases where an object was projected further than what the *identity threshold* allowed, due to noise, leading in considering it as two different objects.

Finally, depending on whether the objects are dynamic or not, were sent into the predictions module to predict their future trajectory and avoid collisions with them. A demonstration of the 3D semantic mapping on the dynamic map can be seen in figure 8.

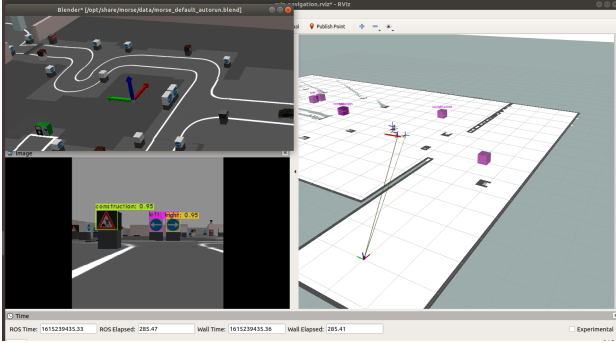


Fig. 8: 3D Objects Semantic Mapping

C. Prediction

Online prediction - Recurrent Neural Networks (RNN)

We call dynamic obstacles those objects that are moving in the environment and that an autonomous vehicle must avoid while predicting their future trajectory. The base approach we are taking is the one proposed in [17] for real-time dynamic obstacle motion prediction. The model received as input recent observations of the obstacles perceived by the car, then it trains and predicts trajectories in future time steps and outputs the corresponding estimated poses. We decided to tackle this task without using any extensive datasets, given that it would be a much more complex to build and pre-process solution, when we only have a limited number of dynamic obstacles. Input data for motion prediction is quite specific and detailed, and for our scenario that would be over-skilled, in addition to

the computational costs that running a big neural network for prediction implies, and that's a constraint in our case.

The prediction model is adapted from the original implementation in [34] to have it integrated with our robot application on ROS Melodic. TensorFlow [35] is the machine learning platform used behind to build and run the predicting RNNs. A Gaussian Process framework, GPy [36], is also part of the package stack.

The predictor, as we call this model, is compound of two Long Short-Term Memory (LSTM) networks, one for training and the other one for the actual prediction (figure 9). The reason for this is that dropout sampling is applied in both to approximate a probability distribution and incorporates uncertainty, but the criteria varies in each case: the training network always uses different masks at every timestep for regularization, whereas the prediction network uses one same mask to approximate a Bayesian network. Each dynamic obstacle in the environment requires its own predictor, for which we had to adjust our code so that it is generic and scalable to deal with multiple agents separately and simultaneously.

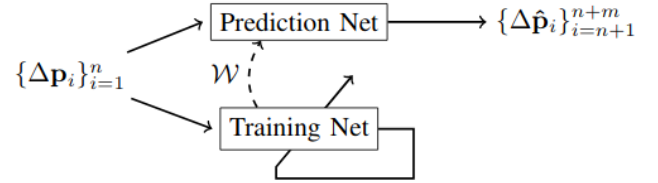


Fig. 9: Online predictor with two LSTM networks running in parallel.

D. Motion Planning

1) **Local and Global Planning:** In the robotics domain, we generally have a global planner and a local planner for motion planning. The global planner lays out the plan based on a global costmap, whereas the local planner tries to optimize and follow the global plan by creating a local plan based on a local costmap.

For our project, we use the global planner of the *move_base* ros package, which uses the underlying Dijkstra algorithm for path planning. Furthermore, for the local path planning, we rely on Time-Elastic-Band (TEB) local planner. It optimizes the robot's trajectory concerning trajectory execution time, separation from obstacles, and compliance with kinodynamic constraints at runtime [37]. A large set of parameters had to be tuned for the local planner, as per the simulation environment concerning the road lanes nature and curvature. These parameters are listed in *teb_local_planner_simu.yaml* file and can be modified as per system requirements.

2) **Dynamic Mapping:** The dynamic map is the most crucial component of our project. The dynamic map provides

information about the car's environment to help it plan paths securely and efficiently on the map. We followed a layered 2d costmap structure to create a dynamic map. The layered structure is shown in figure 10. The static, obstacle and inflation layers are provided with the packages, and our challenge was to create the lane and moving obstacle layer. For the lane layer, we project the detected lanes on an occupancy grid map and publish it. This map is then subscribed by the lane layer plugin and then integrated with the master costmap.

Apart from the 2d costmap, we also have a semantic map where we visualize the detected objects in 3d by placing them on the map in rviz. This information helps the behaviour planner know how far the detected objects are from the car. For example - the stop sign may be detected by the perception module even when the car is far away, but it is logical for the car to stop only when it is close to the stop sign.

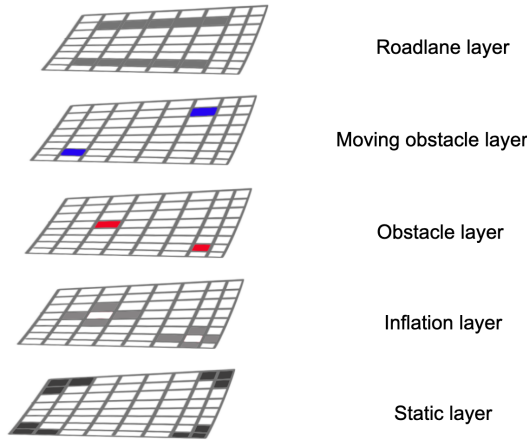


Fig. 10: Dynamic map with a layered cost structure. There are five layers (from top to bottom) - lane, moving obstacle, static obstacle, inflation and static layers.

E. Decision Making

1) **Behavioural Planning:** The primary output of the behavior planner is a driving maneuver to be performed in the current environment. Along with the driving maneuver, the behavior planner also outputs a set of constraints, which constrain the local planning problem. The constraints which we will use and populate throughout this report include, the default path from the current location of the vehicle to the destination goal, for many behaviors, it may be the charging or parking station or anywhere else in the simulation environment. It also includes maintaining the center line of the current lane and the speed limit along the default path. The lane boundaries of the current lane should be maintained under nominal driving conditions. Any future stop locations which the vehicle needs to arrive at and this constraint is only populated if the relevant maneuver is selected. Finally, the set of dynamic objects of high interest which the local planner should attend to. These dynamic objects may be important

due to proximity or estimated future path [38].

One approach traditionally used to represent the set of rules required to solve behavior selection is a finite state machine [39]. The first set of components of a finite state machine is the set of states. For the behavior planning system, the states will represent each of the possible driving maneuvers, which can be encountered. In our environment, we will only need four possible maneuvers or states:

- **Driving:** given goal not achieved
- **Idle:** goal achieved or no new goal point
- **Recharging:** going to charging station when the battery is low
- **Parking:** park the car at the parking area after detecting an empty spot

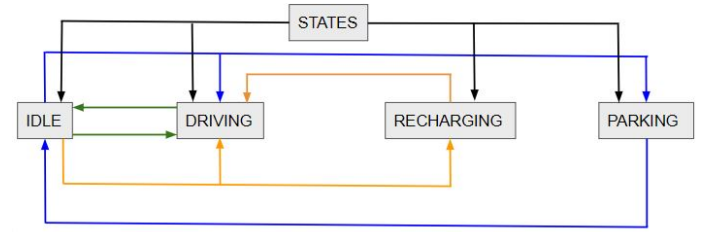


Fig. 11: Finite states of behavioural planner

The maneuver decision defined by the behavior planner is set by the states of the finite state machine. Each state has associated with it an entry action, which is the action that is to be taken when a state is first entered. For our behavior planner, these entry actions involve setting the necessary constraint outputs to accompany the behavior decision. For instance, as soon as we enter the driving state, we must also define the stopping point along the path (which is the final goal in our case). Similarly, the entry condition for the idle state sets the final goal to be achieved. The second set of components of the finite state machine are the transitions, which define the movement from one state to another. In our four-state example, we can perform the following transitions:

- Idle to Driving and Driving to Idle
- Idle to Driving to Recharging and Recharging to Idle to Driving
- Idle to Driving to Parking and Parking to Idle

There can also be transitions that return us to the current state, triggering the entry action to repeat for that state. Each transition is accompanied by a set of transition conditions that need to be met before changing to the next state. These transition conditions are monitored while in a state to determine when a transition should occur. For example, the transition conditions going from idle to driving state requires the action of providing a final goal to the robotic car. Similarly, the transition condition for going to the recharging state is checking if the battery is above or below the threshold value and then driving to the charging station. This four-state example highlights the most

important aspects of the finite state machine-based behavior planner.

2) **Parking**: One of the use case scenarios and goal of the decision-making module is to find the parking spot by providing the location of the parking station (parking entrance) or by exploring in the simulation environment. In the case of providing the destination of the parking station in terms of the pose on the map, the car first tries to go to the destination point of the parking station. Then it tries to detect the empty spots in the available field-view of the camera. If an empty parking slot is detected, the state is changed from driving to parking, and the car parks itself autonomously at that empty spot. In the case of exploration, the car would explore the map in the simulation environment to find the parking entrance by itself, unlike the previous case. We managed to implement the parking behavior by providing the pose of the parking station on the map as the goal point.

For the parking case, the implementation of the parking state is included as behavior or state in the finite state machine. The empty parking spots are being detected by the YOLO perception module. The center image coordinates of parking spots are converted to world coordinates using depth information and camera parameters. Finally, the computed pose is reached with help of global and local planners.

V. RESULTS AND DISCUSSION

In this section, we categorize the work done in each sub module on the basis of experimentation, description, analysis and then a possible discussion.

A. Localisation and Mapping

We did an experiment to measure the performance of the localization module. The experiment comprises of comparing the pose estimate of the car with its ground truth. We calculated the position error in x, y directions and orientation error in the z-direction(yaw) to compare and tune the parameters for robot localization and mapping. The experiment was conducted with multiple trials. In each trial, the car had to reach a given goal point. The ground truth pose (from Morse environment) and the estimated pose (from filtered odometry) were recorded and analyzed for each trial.

Test Cases	Distance (m)	R.M.S.E (m)	M.S.E (m ²)	M.D.E (m)	M.A.E (m)
Test 1	9.47	0.0486	0.00236	0.1554	0.0481
Test 2	8.29	0.0777	0.00605	0.2808	0.0664
Test 3	11.37	0.0875	0.00764	0.3238	0.0858
Test 4	23.66	0.0703	0.00495	0.3459	0.0597
Test 5	15.58	0.0731	0.00534	0.5353	0.0561
Average	13.67	0.0714	0.00527	0.328	0.0632

TABLE II: Evaluation matrix for odometry pose error: Position (X and Y)

Test Cases	Distance (m)	R.M.S.E (rad)	M.S.E (rad ²)	M.D.E (rad)	M.A.E (rad)
Test 1	9.47	0.00452	0.000021	0.011	0.003763
Test 2	8.29	0.01272	0.000162	0.0647	0.008796
Test 3	11.37	0.0168	0.000281	0.0918	0.009854
Test 4	23.66	0.00738	0.000055	0.0631	0.004638
Test 5	15.58	0.04598	0.002114	1.9999	0.007843
Average	13.67	0.0221	0.000526	0.4461	0.006978

TABLE III: Evaluation matrix for odometry pose error: Orientation (Yaw)

Tables II and III show the Root mean square error (R.M.S.E), Mean square error (M.S.E), Maximum distance error (M.D.E), and Maximum absolute error (M.A.E) values calculated on different paths for different trials. All methods presented provide positive values, and a smaller value is considered better.

Mean Square Error is a simple and common metric error estimator for regression analysis. MSE will treat the errors as exponential, meaning that an error of 2m will be considered four times as bad as an error of 1m. Thus making this method sensitive to outliers. Mean Absolute Error is widely used in finance due that all individual differences are weighted equally in the average. MAE is less sensitive to outliers than MSE since the MAE will provide a linear score of the errors, i.e., an error of 2m will be considered twice as bad as an error of 1m. Root Mean Square Error is the square root of MSE, making them somewhat similar. RMSE is perhaps a more intuitive method to use since it describes the average distance between two data set points. MDE is defined by taking the maximum value of the distance between two sets of points. Hence, we decided to go with RMSE for calculating the estimated pose error.

The results for the localization show that after several trials of testing, the hector_slam seems to perform reasonably well in the simulation environment with an error of 0.07 m in position and 0.0221 rad (1.26°) in orientation. It would be really exciting to visualize the performance and accuracy of hector_slam with gmapping [40] which utilizes a Rao-Blackwellized particle filter and accepts odometry information along with LiDAR scans. Competitive analysis between both the SLAM algorithms using different motion models opens a topic of potential research on mapping of robotic cars in an indoor environment.

B. Perception

1) **Object Detection**: We trained our YOLOv4 model for 22,000 iterations with 17,600 and 19,800 steps, batches of size 64 and further subdivision of 16. It had a learning rate of 0.001 and it was trained on CUDA/CuDNN. The best results for our simulation occurred at around 19,000 iterations. After that the network resulted in worse results due to overfitting. Thus, for the final evaluation we used the weights of the 19,000 iterations.

To evaluate the performance of the YOLOv4 trained model we calculated the mean average precision, precision, recall, F1-score and the average IoU. The results for IoU for 50% and 75% can be seen in the table IV.

	IoU=50	IoU=70
mAP	0.91	0.77
Precision	0.93	0.87
Recall	0.99	0.92
F1-Score	0.96	0.89
Average IoU	0.80	0.75

TABLE IV: Evaluation matrix of YOLOv4 Tiny

We noticed that the best results for IoU=50 occurred at different iteration compared to IoU=75. However we chose to optimize the IoU=50 results.

The results of YOLOv4 were particularly promising, outperforming the SSD model, and that was the reason we chose to further develop and implement the YOLOv4 model in our autonomous vehicle. At the same time, the integration of YOLOv4 using the *darknet_ros* package performed with minimum latency, making it appropriate for the needs of this project, even though latency highly depends on the computational power of each machine.

2) **Lane Detection and Segmentation:** The *LaneNet* model was trained for 15000 iterations with a learning rate of 0.001. At this point, the model did not improve further. The accuracy of the binary segmentation is 0.99. The value of the discriminative loss function of the binary segmentation is 0.06. For the instance segmentation it is 0.08. The course of these values during training can be seen in figure 12.

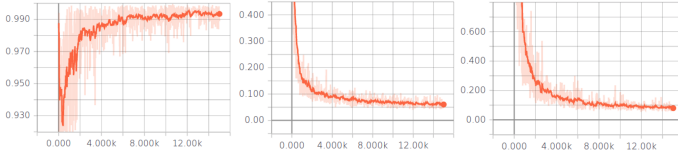


Fig. 12: Accuracy, binary segmentation loss, and instance segmentation loss of LaneNet model

The instance segmentation did not work as expected when using it in the real environment. Therefore, we decided to only use the binary segmentation result for our implementation of the lane detection node.

For the integration of the produced global costmap into a layered costmap that can be used by our planner we were required to use the same map metadata (e.g. resolution) as the already existing maps. The map produced by the *LaneNet* node after the robot was driving around the whole track can be seen in figure 13.

The map presented in figure 13 does not look perfect. For example, as described in the methodology, there are small gaps in the lanes. The planner may use this gaps to escape the lanes and therefore it is important to solve this issue. The attempt to

tackle that problem with curve fitting was not successful. A lot of the times, points get connected by straight lines although they should not be connected. This happens also the other way around. Also, in complicated surroundings like turns and intersections it can happen that too many map cells become occupied. When this happens, the planner is not able to find a path though this place any more.

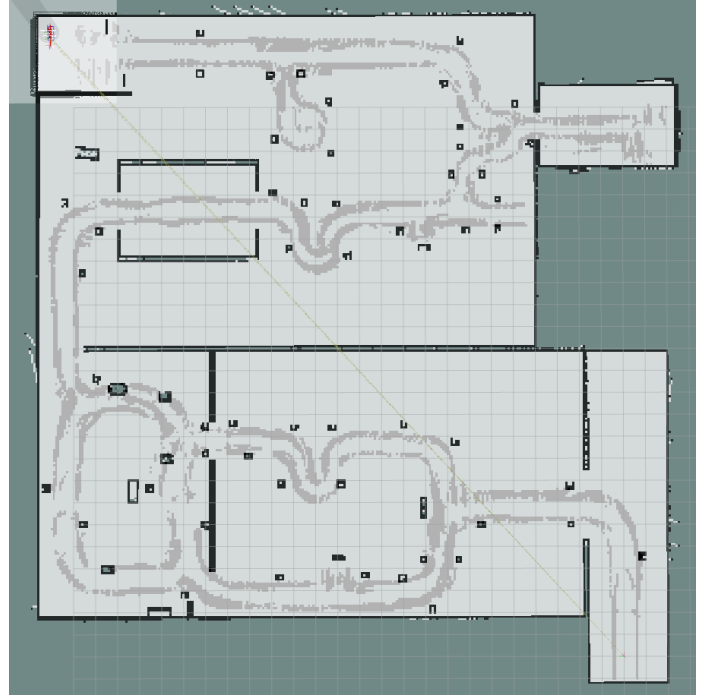


Fig. 13: Final lane costmap generated by the lane detection node drawn on the real map

As described in chapter IV-B2, we were able to test different thresholds for our implementation. We could achieve the best results using a hit threshold of 40, a hit ratio threshold of 0.6 and a buffer size of 500. We also project only points to the map that are nearer to the camera than the maximum depth of 4. Besides the restrictions of the layered costmap, we noticed that we can achieve a lot better results by scaling up the map resolution. Since it was not possible for us to integrate such a map into our layered costmap, we use the same metadata for the lane costmap as the other layers. Also, it should be noted that the *LaneNet* model cannot detect lanes in real time on our computers. Thus, we only process every 15th camera image.

3) **3D Objects Semantic Mapping - Tracking:** To evaluate the performance of the 3D objects semantic mapping we compared the objects that had been projected into the dynamic map, with the ground truth of the YOLOv4 detected objects. Here we explicitly mention the importance of **tracking**, since it is the main reason for false positives and false negatives in our experiments.

For evaluating the accuracy of this module, we chose to use precision and recall formula adjusted for the needs of

the objects projector. Thus, since precision takes into account false positives, it gives us a hint about the performance of the module against occlusions. On the other hand, recall gives us a hint about noise of the transformation of the depth's point cloud in the dynamic map, since false negatives refer to objects that falsely were projected in a location far from the ground truth. Therefore in this case, precision and recall also resembles **tracking** efficiency.

We conducted the experiments for a total of 100 different detected objects, for *identity thresholds* of 0.5, 1, 2, 3 and 4 meters. The results of precision and recall for different *identity threshold* values are shown in Table V.

Identity Threshold (m)	Precision	Recall
0.5	0.96	0.86
1.0	0.95	0.88
2.0	0.95	0.90
3.0	0.94	0.93
4.0	0.93	0.95

TABLE V: Evaluation of 3D objects semantic mapping

By Table V we conclude that as precision increases, recall decreases, i.e. the less the occlusions in the dynamic map, the more the duplicated objects. However, for the needs of this simulation environment it becomes clear that occlusions are rarely occurred since objects of the same class have relatively large distance in-between them, compared to the projection noise. Thus, we chose the *identity threshold* of 4m for our final integration.

Last, even though the accuracy values are relatively high, the noise accumulates from the object detection and into the prediction module, resulting in undesirable results like false projections. Thus, ideally we should use a more sophisticated tracking algorithm. For this purpose, we tried integrating **deep-sort** algorithm through [41] or **Extended Kalman Filter** through [42] fused with the YOLOv4 model, however the complexity for doing so, rose substantially compared with the simplicity of the simulation, but also since we didn't included it in the initial task allocation phase. That was the main reason for choosing to integrate the simple tracking approach as was described above. As a take-away from this experimentation, we realised the importance of a tracking module for the object detection task, as well as to optimize it in order to meet the needs of the computational power of the simulation.

C. Prediction

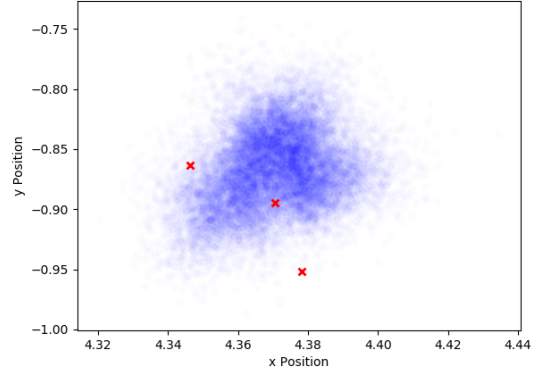
The three main challenges when building the prediction module for the robot are:

- Tune for multiple obstacles previously unknown
- Handle latency in topic messages publishing
- Unique object predictor

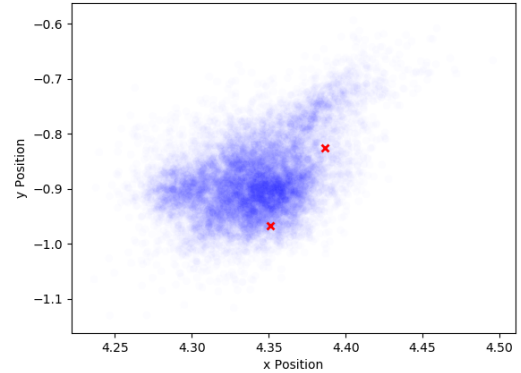
We succeed at overcoming those issues and had independent predictors per dynamic obstacles, taking care of unique identi-

fiers and synchronization between publishers from perception and subscribers on prediction.

To test the performance of our prediction module, we took an obstacle detected with the autonomous vehicle as an example and compared the actual pose after M time steps against the predicted poses at that instant. Figure 14 shows the estimated future position of the obstacle remains within really small error measurements, which enables us to have a feasible performance for the prediction block in the autonomous robot.



(a) Sample 1



(b) Sample 2

Fig. 14: Comparison of dynamic obstacle's actual vs predicted poses in time.

The blue spots depict the estimated outputs of the prediction model and the red marks the real pose of the object in the environment. Only a very small difference of about 0.05 units for the coordinates in X and Y is present.

Our approach appeared to be the most feasible for our use case, since not many heterogeneous moving obstacles were present in the simulation environment and an over-skilled extensive deep learning training with a complex dataset was not suitable. Moreover, the computational resources are an inevitable limitation of the team for trying out bigger models. The results in the end showed to be reliable with the RNN model implemented, since the predicted trajectories describe

accurately the real pose of the dynamic obstacles throughout time based on recent observations. The poses are projected in the cost map for the local planner to react in order to avoid the objects.

D. System Integration

We managed to integrate object detection, lane detection and segmentation, dynamic map, motion planning, and decision making together to constitute our final system. We did the following four experiments to test our system.

- Allowed the car at rest to see its behavior when the battery charge goes below a threshold. Ideally, the car should go to the charging station and recharge itself.
- Created a scenario where the battery discharges below a threshold while chasing a goal point. The car is supposed to prioritize the recharging and then continue to the original goal point.
- Set the goal point to the parking station with an objective of parking during the start of the simulation.
- Taking the car manually close to the parking station and setting parking as a goal.

Metric	Experiment no.			
	a	b	c	d
Trials	20	2	10	5
Collisions with static objects	0	0	0	0
Collisions with dynamic objects	0	0	0	0
Correctly classified objects	10/10	4/4	5/5	11/11
Semantically mapped objects	9/9	4/4	5/5	7/8
Stayed within the lanes	2/20	1/2	0/10	1/5
Successful accomplishment of the goal	2/20	1/2	0/10	3/5

TABLE VI: Experimental results

The experiment's results are summarized in Table VI with specified metrics. Each experiment was conducted for a different number of trials based on its complexity and obtained results. There are no collisions with the static objects for each experiment as they are well projected on the costmaps using 2D Lidar and SLAM. The dynamic obstacles are not encountered in any of the experiments, and even if they did, there is a very rare chance of collision (despite not having a prediction costmap) as those objects have simple trajectories.

The object detection and semantic mapping modules perform as per the obtained metrics. The classification accuracy is around 100%, and almost all of them are semantically mapped on rviz using Markers. Staying within the lanes was challenging in all of the experiments. Due to the discontinuity in the lanes, the motion planners always found a way to make the car travel through those gaps, as shown in Figure 15. The successful completion of the goal for each experiment is directly related to the lane crossing. If the lanes were not crossed, the car could easily reach the goal point, but if it left the lane limits, it was hard for planners to get back on track.

For experiment c), where the car had to travel a long distance to park, it failed each time it reached the tri-junction as shown in Figure 16. As the projected lanes get very wide, the planners failed to find a viable path, and they restart, causing the costmaps to disappear and ruining the simulation. Especially for experiment d), where the car is manually taken close to the parking station and asked to park, the car was able to park despite not keeping within the lanes a couple of times. It is due to the relatively simple lane structure (no lane curves).

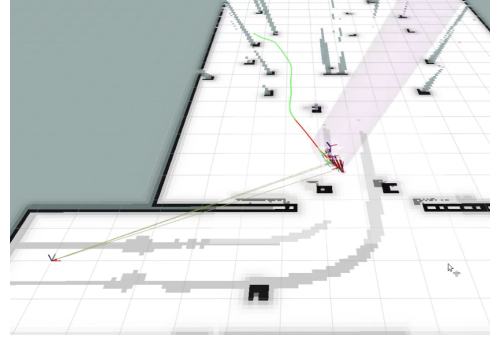


Fig. 15: Car going through the gaps in between the lanes.

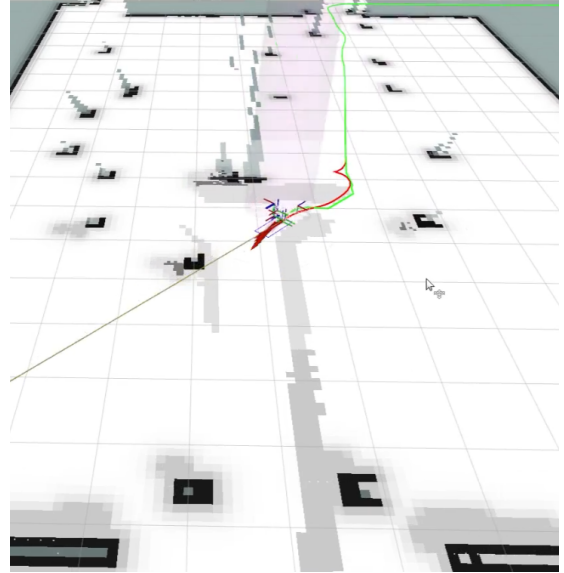


Fig. 16: Motion planners failing to find the feasible path to follow due to very thick lane projections.

As part of the discussion, we would like to implement a robust regression method for the lanes to be thin and continuous. We did try some methods, but it did not work because of the high complexity, as ideally, we need to separate the right, left and straight lanes at tri-junction and then apply regression individually to all of them. The primary reason for lanes to be thick and discontinuous is the low resolution of the camera images. Also, since we had to run the inference

on a GPU, it made the whole process very slow, and the fps was drastically reduced.

VI. CONCLUSION AND FUTURE WORK

Our perception module - object detection using *Yolo v4* architecture and lane detection and segmentation using *LaneNet* architecture has excellent performances in the simulation environment. The motion planning and decision-making based on finite-state-machine also work as expected. There are quite a few challenges in creating the dynamic map, especially with the *lane costmap* layer.

Consequently, the car can navigate autonomously to close goal points (in a radius of 12 meters) following the constraints but failed at far distances. The bear car can also park itself autonomously if given the parking station's location and vicinity very close to the parking station. The main reason for failing is the lane projection on the costmap not working robustly at tri-junction and sharp turns. At those challenging locations, the projected lanes either cover the whole road, and the planners could not find a feasible path to the goal point, or there are significant gaps left on the lanes such that the car crosses the lanes.

Therefore, as part of future work, the lane costmap has to be improved with better curve fitting techniques. Also, lane segmentation is computationally expensive and requires a high-end graphics card to run smoothly. Hence, other efficient and robust lane detection techniques could also be explored.

Furthermore, integration of data-driven methods for tracking proved to be computationally expensive for our simulation's needs; thus, for more accurate tracking, we could use and optimize the *deep-sort* algorithm, particularly for the case of our autonomous car.

Additionally, for representing a more realistic scenario on the concept of autonomous driving, we could integrate an exploration module to find the desired destination point, in this case, the parking spot, instead of giving it a goal location. Finally, the predicted trajectories of moving objects will have to be projected on the dynamic map to account for moving obstacles.

ACKNOWLEDGEMENT

We are extremely grateful to our guide and supervisor, Yuan Xu, a Postdoctoral Researcher at TU Berlin to support and encourage us. He patiently went over the drafts suggesting numerous improvements, and constantly motivated us in our project work. We would also like to thank Orhan Can Görür, Postdoctoral Researcher at DAI Labor, TU Berlin, to motivate us and expedite the process of solving issues during the course work.

REFERENCES

- [1] S. Schaal, "Robots are becoming autonomous." [Online]. Available: <https://www.mpg.de/9271075/annual-report-2014-schaal.pdf>
- [2] G. Bresson, Z. Alsayed, L. Yu, and S. Glaser, "Simultaneous localization and mapping: A survey of current trends in autonomous driving," *IEEE Transactions on Intelligent Vehicles*, vol. 2, no. 3, pp. 194–220, 2017.
- [3] R. Mur-Artal and J. D. Tardos, "Orb-slam2: An open-source slam system for monocular, stereo, and rgb-d cameras," *IEEE Transactions on Robotics*, vol. 33, no. 5, p. 1255–1262, Oct 2017. [Online]. Available: <http://dx.doi.org/10.1109/TRO.2017.2705103>
- [4] J. Tang, L. Ericson, J. Folkesson, and P. Jensfelt, "Gcnv2: Efficient correspondence prediction for real-time slam," 2019.
- [5] J. Redmon, S. K. Divvala, R. B. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," *CoRR*, vol. abs/1506.02640, 2015. [Online]. Available: <http://arxiv.org/abs/1506.02640>
- [6] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "Ssd: Single shot multibox detector," *Lecture Notes in Computer Science*, p. 21–37, 2016. [Online]. Available: http://dx.doi.org/10.1007/978-3-319-46448-0_2
- [7] E. Yurtsever, J. Lambert, A. Carballo, and K. Takeda, "A survey of autonomous driving: Common practices and emerging technologies," *IEEE Access*, vol. 8, p. 58443–58469, 2020. [Online]. Available: <http://dx.doi.org/10.1109/ACCESS.2020.2983149>
- [8] G. Liu, F. Wörgötter, and I. Markelic, "Combining statistical hough transform and particle filter for robust lane detection and tracking," 07 2010, pp. 993 – 997.
- [9] "Culane," <https://xingangpan.github.io/projects/CULane.html> (last accessed on 16/03/21).
- [10] "Tusimple," <https://github.com/TuSimple/tusimple-benchmark/wiki> (last accessed on 16/03/21).
- [11] "Caltech," <http://www.mohamedaly.info/datasets/caltech-lanes> (last accessed on 16/03/21).
- [12] "Bdd," <https://bair.berkeley.edu/blog/2018/05/30/bdd/> (last accessed on 16/03/21).
- [13] S. Lee, J. Kim, J. S. Yoon, S. Shin, O. Bailo, N. Kim, T.-H. Lee, H. S. Hong, S.-H. Han, and I. S. Kweon, "Vpnet: Vanishing point guided network for lane and road marking detection and recognition," 2017.
- [14] D. Neven, B. D. Brabandere, S. Georgoulis, M. Proesmans, and L. V. Gool, "Towards end-to-end lane detection: an instance segmentation approach," *CoRR*, vol. abs/1802.05591, 2018. [Online]. Available: <http://arxiv.org/abs/1802.05591>
- [15] T. Salzmann, B. Ivanovic, P. Chakravarty, and M. Pavone, "Trajectron++: Multi-agent generative trajectory forecasting with heterogeneous data for control," *CoRR*, vol. abs/2001.03093, 2020. [Online]. Available: <http://arxiv.org/abs/2001.03093>
- [16] J. Houston, G. Zuidhof, L. Bergamini, Y. Ye, A. Jain, S. Omari, V. Iglovikov, and P. Ondruska, "One thousand and one hours: Self-driving motion prediction dataset," <https://level5.lyft.com/dataset/>, 2020.
- [17] V. Kurtz and H. Lin, "Toward verifiable real-time obstacle motion prediction for dynamic collision avoidance," *CoRR*, vol. abs/1811.01075, 2018. [Online]. Available: <http://arxiv.org/abs/1811.01075>
- [18] D. González, J. Pérez, V. Milanés, and F. Nashashibi, "A review of motion planning techniques for automated vehicles," *IEEE Transactions on Intelligent Transportation Systems*, vol. 17, no. 4, pp. 1135–1145, 2016.
- [19] M. Foukarakis, A. Leonidis, M. Antona, and C. Stephanidis, "Combining finite state machine and decision-making tools for adaptable robot behavior," in *HCI*, 2014.
- [20] M. Olsson, "Behavior trees for decision-making in autonomous driving," Master's thesis, KTH Kungliga Tekniska Högskolan, Stockholm, Sweden, 2016.
- [21] H. Durrant-Whyte and T. Bailey, "Simultaneous localization and mapping: part i," *IEEE Robotics Automation Magazine*, vol. 13, no. 2, pp. 99–110, 2006.
- [22] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics (Intelligent Robotics and Autonomous Agents)*. The MIT Press, 2005.
- [23] S. Kohlbrecher, O. V. Stryk, T. U. Darmstadt, J. Meyer, and U. Klingauf, "A flexible and scalable slam system with full 3d motion estimation," in *International Symposium on Safety, Security, and Rescue Robotics*. IEEE, 2011.
- [24] K. Wada, "Label me," github.com/wkentaro/labelme (last accessed on 15/03/21).
- [25] "Tensorflow model detection zoo." [Online]. Available: https://github.com/tensorflow/models/blob/master/research/object_detection/g3doc/tf1_detection_zoo.md

- [26] "Tensorflow object detection api." [Online]. Available: https://github.com/tensorflow/models/tree/master/research/object_detection
- [27] "Robotic systems lab - legged robotics at eth zürich," github.com/leggedrobotics/darknet_ros (last accessed on 15/03/21).
- [28] Alexey, "Scaled-yolov4 and yolov4," github.com/AlexeyAB/darknet (last accessed on 15/03/21).
- [29] J. Redmon, "Darknet," github.com/pjreddie/darknet (last accessed on 15/03/21).
- [30] "lanenet-lane-detection," <https://github.com/MaybeShewill-CV/lanenet-lane-detection> (last accessed on 16/03/21).
- [31] "Lanenetros," <https://github.com/AbangLZU/LaneNetRos> (last accessed on 16/03/21).
- [32] "freenect_launch - ROS Wiki." [Online]. Available: http://wiki.ros.org/freenect_launch
- [33] "GitHub - verlab/3DSemanticMapping_jint_2020: Repository for the paper "Extending Maps with Semantic and Contextual Object Information for Robot Navigation: a Learning-Based Framework using Visual and Depth Cues"." [Online]. Available: https://github.com/verlab/3DSemanticMapping_JINT_2020
- [34] "Rnncolloid," https://github.com/vincekurtz/rnn_colloid.
- [35] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015, software available from tensorflow.org. [Online]. Available: <https://www.tensorflow.org/>
- [36] GPY, "GPY: A gaussian process framework in python," <http://github.com/SheffieldML/GPY>, since 2012.
- [37] "Teb local planner," http://wiki.ros.org/teb_local_planner (last accessed on 15/03/21).
- [38] C. Hubmann, M. Becker, D. Althoff, D. Lenz, and C. Stiller, "Decision making for autonomous driving considering interaction and uncertain prediction of surrounding vehicles," 06 2017, pp. 1671–1678.
- [39] M. Foukarakis, A. Leonidis, M. Antona, and C. Stephanidis, "Combining finite state machine and decision-making tools for adaptable robot behavior," in *Universal Access in Human-Computer Interaction. Aging and Assistive Environments*, C. Stephanidis and M. Antona, Eds. Cham: Springer International Publishing, 2014, pp. 625–635.
- [40] "gmapping," <http://wiki.ros.org/gmapping> (last accessed on 15/03/21).
- [41] "Tracker ros node (sort and deep sort) using darknet_ros (yolov3)." [Online]. Available: <https://github.com/ilyasmg/sort-deepsort-yolov3-ROS>
- [42] "Yolo ros: Real-time object detection for ros." [Online]. Available: https://github.com/pushkalkatara/darknet_ros